Serial LCD Controller (*Roulette Game*)
Laboratório de Informática e Computadores 2024 / 2025 verão
Autores: Margarita Sysoeva 51403 / Pilar Gonçalves 51806

O módulo *Serial LCD Controller (SLCDC)* implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 1.

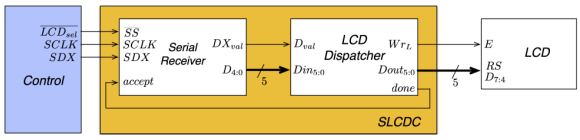

Figura 1: Diagrama de blocos do módulo *SLCDC*

O módulo *SLCDC* recebe em série uma mensagem constituída por cinco (5) bits de informação e um (1) bit de paridade. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura 2, em que o bit RS é o primeiro bit de informação e indica se a mensagem é de controlo ou dados. Os seguintes quatro (4) bits contêm os dados a entregar ao *LCD*. O último bit contém a informação de paridade ímpar, utilizada para detetar erros de transmissão.

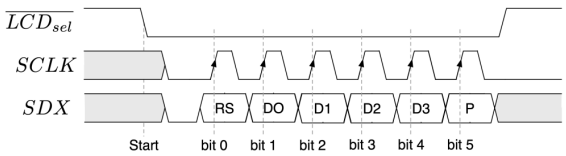

Figura 2: Protocolo de comunicação com o módulo *SLCDC*

O emissor, realizado em *software*, quando pretende enviar uma trama para o módulo *SLCDC* promove uma condição de início de trama (*Start*), que corresponde a uma transição descendente na linha ($\overline{LCD_{sel}}$). Após a condição de início, o módulo *SLCDC* armazena os bits de dados da trama nas transições ascendentes do sinal *SCLK*.

1 Serial Receiver

O bloco **Serial Receiver** do módulo *SLCDC* é constituído por quatro blocos principais: i) um bloco de controlo; ii) um bloco conversor série paralelo; iii) um contador de bits recebidos; e iv) um bloco de validação de paridade, designados por *Serial Control*, *Shift Register*, *Counter* e *Parity Check* respetivamente. O bloco *Serial Receiver* deverá ser implementado com base no diagrama de blocos apresentado na Figura 3.

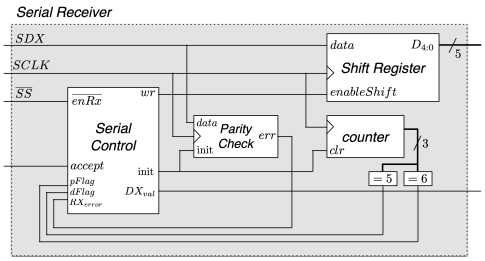

Figura 3: Diagrama de blocos do bloco *Serial Receiver*

O bloco **Serial Control** foi implementado através de uma máquina de estados sequencial, representada na Figura 4. Esta máquina coordena a receção dos bits da trama, o controlo do *Shift Register*, a verificação de paridade e a validação final do dado recebido.

A máquina inicia no estado **INITIAL_STATE**, onde o sinal *init* é ativado para reiniciar o contador. Permanece neste estado até que *enRx* = '1', indicando o início da receção.

Passa então ao estado **RECEIVING_STATE**, onde o sinal *wr* é ativado para registar o bit de entrada. Este processo repete-se enquanto *enRx* = '1' e *dFlag* = '0', ou seja, enquanto não tiverem sido recebidos os quatro (4) bits principais da trama.

Quando *dFlag* = '1', a máquina avança para o estado **PARITYCHECK_STATE**. Neste estado, o contador continua a avançar até o sinal *pFlag* = '1', indicando que o 5.º bit (bit de paridade) foi recebido, caso contrário, transita para o INITIAL_STATE, e se o sinal *enRX* = '1', a máquina permanece no estado PARITYCHECK_STATE.

Em seguida, verifica-se o sinal *RXerror*. Se for '1', indicando erro de paridade, a máquina volta ao INITIAL_STATE. Caso contrário, transita para o estado **SENT_STATE**, ativando o sinal *DXval* para informar que foi recebida uma trama válida.

A máquina permanece em SENT_STATE até que o sinal *accept* = '1', momento em que passa para o estado **WAITACCEPT_STATE**. Este estado garante que a máquina só regressa ao estado inicial depois de *accept* voltar a '0', evitando reenvios ou processamento duplicado.

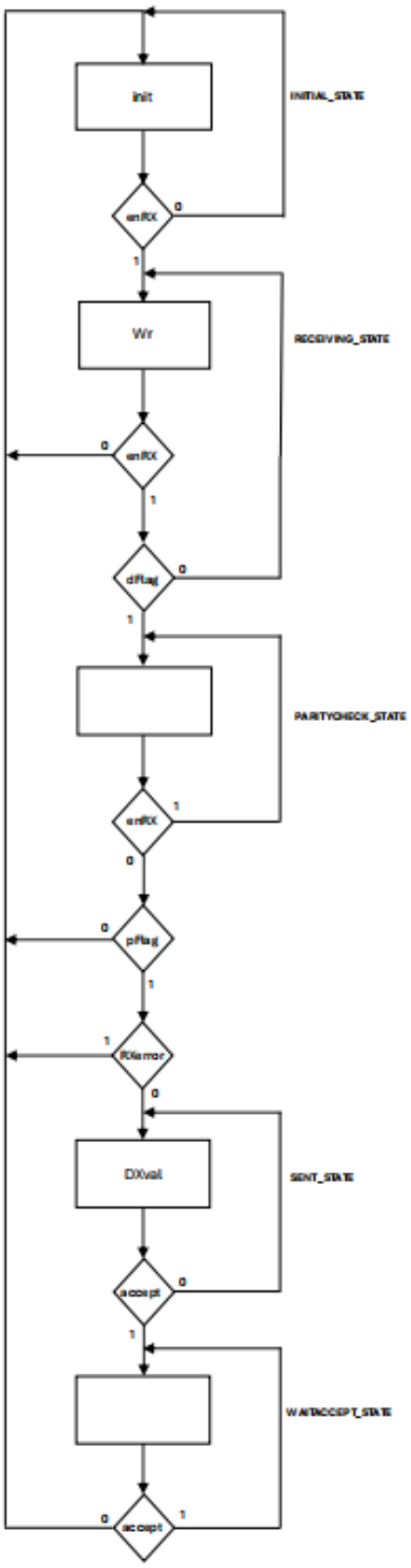

Figura 4: Máquina de estados do módulo *Serial Control*

O **Shift Register** armazena os bits de dados à medida que são recebidos em série através de *SDX*, sincronizados com os flancos do sinal *SCLK*. Após a receção dos cinco (5) bits, o conteúdo do registo é transferido para o *LCD*.

O **Counter** é incrementado a cada ciclo de receção e serve para controlar o número de bits recebidos. Está ligado a dois comparadores que sinalizam quando foram recebidos seis (6) ou cinco (5) bits (com e sem paridade, respetivamente).

Por fim, o módulo de **Parity Check** calcula a paridade dos bits recebidos e compara com o bit de paridade transmitido. Se for detetado erro, o sinal *RXerror* é ativado e o valor não é enviado para o *LCD*.

2 LCD Dispatcher

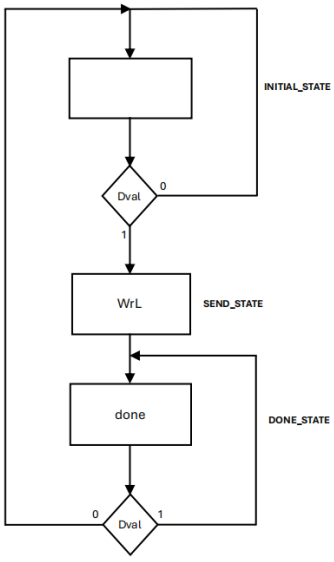

Figura 5: Máquina de estados do módulo *LCD Dispatcher*

O bloco **LCD Dispatcher** (Figura 5) é responsável pela entrega das tramas válidas recebidas pelo bloco *Serial Receiver* ao *LCD*, através da ativação do sinal *WrL*. A receção de uma trama válida é sinalizada pela ativação do sinal *Dval*. O processamento das tramas recebidas pelo *LCD* respeita os comandos definidos pelo fabricante, não sendo

necessário esperar pela sua execução para libertar o canal de receção série. Assim, o bloco *LCD Dispatcher* pode ativar, prontamente, o sinal *done* para notificar o bloco *Serial Receiver* que a trama já foi processada.

3 Control

Implementou-se o módulo *Control* em *software*, recorrendo a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 6.

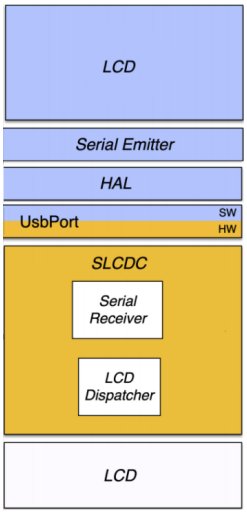

Figura 6: Diagrama lógico do módulo *Control* com a interface *LCD*

3.1 Classe HAL

A classe HAL (*Hardware Abstraction Layer*) foi desenvolvida com o propósito de abstrair a comunicação direta com a porta USB da placa *d10lite*, facilitando a interação com os sinais digitais de entrada e saída do sistema. Esta camada de abstração permite ao programador manipular bits individuais ou conjuntos de bits de forma controlada e consistente, sem recorrer diretamente a operações de baixo nível.

Internamente, a classe mantém uma variável privada *currentState*, inicializada com o valor 0, que representa o estado atual dos bits de saída. Esta variável é atualizada a cada operação de escrita, garantindo que a informação enviada para a porta reflete sempre o estado mais recente dos sinais.

As principais funções implementadas incluem: **init()**: Inicializa o HAL escrevendo o valor inicial (0x00) na porta, garantindo um estado limpo no começo; **isBit(mask: Int)**: Verifica se um determinado bit está ativo (a '1') na leitura da porta; **readBits(mask: Int)**: Retorna os bits de entrada correspondentes à máscara fornecida; **setBits(mask: Int)** e **clrBits(mask: Int)**: Permitem, respetivamente, ativar ou limpar bits específicos, atualizando o estado global do output; **writeBits(mask: Int, value: Int)**: Escreve valores específicos nos bits definidos pela máscara, garantindo que apenas esses bits são alterados.

3.2 Classe SerialEmitter

A classe *SerialEmitter* é responsável por enviar tramas de dados para os blocos físicos que implementam o *SerialReceiver*, como o *SLCDC* (para o LCD) e o SRC (para o display da roleta).

A função principal é **send(addr: Destination, data: Int, size: Int)**, que permite enviar uma trama bit a bit através de *SDX*, sincronizada com pulsos de relógio de *SCLK*. A trama é enviada segundo o protocolo definido no projeto, composta pelos bits de dados e um bit de paridade ímpar no final.

A comunicação inicia-se com a desativação da linha de seleção (*LCD_SEL_MASK* ou *RD_SEL_MASK*) de acordo com o destino (*Destination.LCD* ou *Destination.ROULETTE*). Em seguida, os bits do valor data são enviados da seguinte forma:

Cada bit é extraído e colocado em SDX através de *HAL.setBits(SDX_MASK)* ou *HAL.clrBits(SDX_MASK)*;

Um pulso é gerado no sinal SCLK para validar o bit (*setBits(SCLK_MASK)* seguido de *clrBits(SCLK_MASK)*);

Simultaneamente, é calculado o bit de paridade (*parity*) com *XOR* acumulativo dos bits enviados.

Após o envio dos bits de dados, é enviado o bit de paridade ímpar calculado:

Se *parity* == 0, o bit de paridade é '1', senão é '0', garantindo que o número total de bits a '1' na trama seja ímpar.

Finalmente, o destino é reativado com *HAL.setBits(destinationBit)* para concluir a

transmissão.

Esta abordagem garante uma comunicação síncrona, controlada diretamente em software, sem necessidade de hardware adicional de transmissão, e foi projetada para ser compatível com qualquer *Serial Receiver* implementado em VHDL.

3.3 Classe LCD

A classe LCD controla o visor da *d10lite* através de uma interface de 4 bits, permitindo a escrita de texto e comandos sem que o programador tenha de lidar diretamente com os detalhes do protocolo de comunicação.

O envio de dados é feito com base na constante SERIAL_INTERFACE, que define se a comunicação será feita por interface série (*true*) ou paralela (*false*). Atualmente, o sistema utiliza a interface série.

A comunicação com o LCD é abstraída pela classe *HAL*, responsável por manipular diretamente os bits do *UsbPort*. Foram definidas máscaras associadas aos sinais de controlo:

RS_MASK (bit 4): seleciona entre envio de comando (0) ou dado (1);

E_MASK (bit 5): ativa o LCD para aceitar o nibble enviado;

DATA_MASK (bits 0 a 3): transporta os dados (D4 a D7).

O método **writeNibble(rs: Boolean, data: Int)** determina qual modo de transmissão usar:

writeNibbleParallel(...): implementa o envio em modo paralelo, ativando os sinais RS e E, escrevendo os bits com HAL.writeBits, e desativando o Enable;

writeNibbleSerial(...): codifica os 4 bits de dados e o bit *RS* num total de 5 bits e envia-os usando a classe *SerialEmitter*. Esta função está agora completamente implementada e em uso.

O método **writeByte(rs, data)** envia um byte completo para o *LCD*, dividindo-o em dois nibbles (*MSB* e *LSB*). Este método é chamado tanto por **writeCMD(...)** (para comandos) como por **writeDATA(...)** (para dados).

O método **init()** executa a sequência de inicialização do LCD em modo 4 bits, limpando o ecrã, desligando e reativando o display e configurando o cursor.

Para escrita de conteúdo, a classe disponibiliza:

write(c: Char): escreve um único caractere (ASCII) na posição atual;

write(text: String): escreve uma string completa, com gestão automática da mudança de linha após 16 caracteres e limite a duas linhas;

cursor(line, column): posiciona o cursor em qualquer posição válida (0–1, 0–15), traduzida para os endereços 0x80 e 0xC0;

clear(): limpa o display e reposiciona o cursor;

specialChar(): função preparada para futura extensão com caracteres especiais.

4 Conclusões

O desenvolvimento do módulo *SLCDC* permitiu concretizar a receção de tramas seriais e a comunicação fiável com o *LCD*, validando um protocolo completo de receção, controlo de integridade e envio dos dados.

A implementação em VHDL de blocos como o *Serial Receiver*, *LCD Dispatcher*, *Shift Register*, *Parity Check*, *Counter* e *Comparadores*, em conjunto com a máquina de estados do *Serial Control*, garantiu uma receção sequencial robusta, com verificação de paridade.

No lado do controlo em software, a utilização da linguagem *Kotlin* e a criação das classes *HAL*, *SerialEmitter* e *LCD* demonstraram a eficácia da abstração do hardware através de uma camada de alto nível. Esta abordagem facilitou o envio de comandos e dados para o *LCD*, assegurando a separação entre a lógica de comunicação e a lógica de interface com o utilizador.

Em suma, o projeto cumpriu todos os objetivos propostos, com um sistema capaz de receber tramas série com integridade validada, processá-las corretamente e atualizar o visor *LCD* de acordo com os comandos recebidos.

A VHDL

A.1 SLCDC

```
library ieee;
use ieee.std_logic_1164.all;

entity SLCDC is
        port(
        SS: in std_logic;
        SDX: in std_logic;
        SCLK:in std_logic;
        MCLK: in std_logic;
        RESET: in std_logic;
        WrL:out std_logic;
        Dout: out std_logic_vector(4 downto 0)
        );
end SLCDC;

architecture structural of SLCDC is

component SerialReceiver is
        port(
        SS : in std_logic;
        MCLK : in std_logic;
        SDX : in std_logic;
        RESET : in std_logic;
        SCLK : in std_logic;
        accept : in std_logic;
        D : out std_logic_vector(4 downto 0);
        DXval : out std_logic
        );
end component;

component LCD_Dispatcher is
        port(
        Dval : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
        done : out std_logic;
        Din: in std_logic_vector(4 downto 0);
        Dout: out std_logic_vector(4 downto 0);
        WrL : out std_logic
        );
end component;

signal done_accept : std_logic;
signal Dsignal : std_logic_vector(4 downto 0);
signal Dval_signal : std_logic;

begin
U0: SerialReceiver port map ( MCLK => MCLK, RESET => RESET, SDX => SDX, SCLK => SCLK,
SS => SS, accept => done_accept, D => Dsignal, DXval => Dval_signal );

U1: LCD_Dispatcher port map ( Dval => Dval_signal, Din => Dsignal, RESET => RESET,
CLK => MCLK, done => done_accept, WrL => WrL, Dout => Dout );

end structural;
```

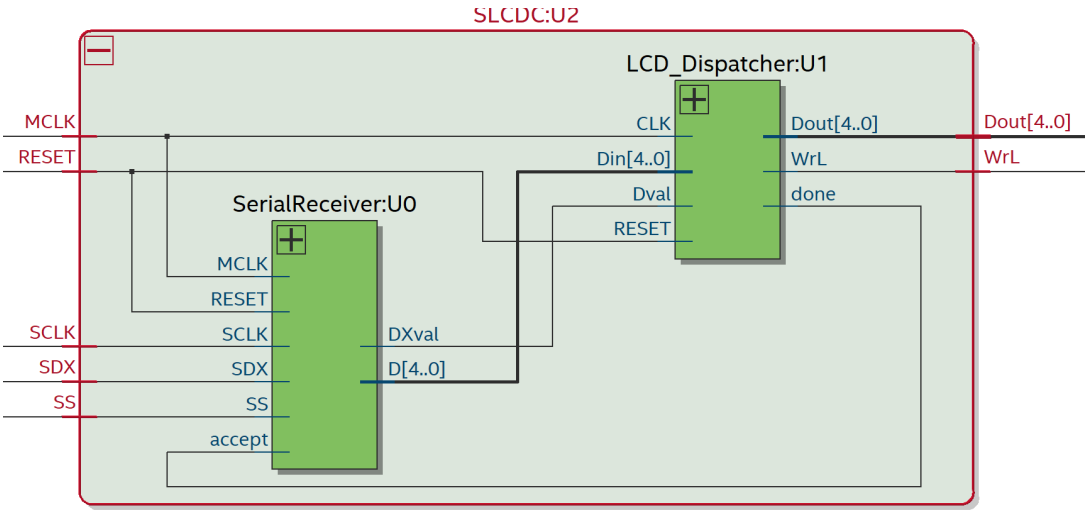

Figura 7: Netlist Viewer do *SLCDC*

A.2 LCD_Dispatcher

```
library ieee;
use ieee.std_logic_1164.all;

entity LCD_Dispatcher is
        port(
        Dval : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
        done : out std_logic;
        WrL : out std_logic;
        Din : in std_logic_vector(4 downto 0);
        Dout : out std_logic_vector(4 downto 0)
        );
end LCD_Dispatcher;

architecture behavioral of LCD_Dispatcher is

type STATE_TYPE is (INITIAL_STATE, SEND_STATE, DONE_STATE);

signal Current_State, Next_State : STATE_TYPE;

begin

Current_State <= INITIAL_STATE when RESET = '1' else Next_State when rising_edge(clk);

GenerateNextState:
process (Current_State, Dval)
        begin
                case Current_State is
                        when INITIAL_STATE =>   if (Dval = '1') then
                                                        Next_State <= SEND_STATE;
                                                else
                                                        Next_State <= INITIAL_STATE;
                                                end if;

                         when SEND_STATE => Next_State <= DONE_STATE;
```

```
                        when DONE_STATE => if (Dval = '1') then
                                                Next_State <= DONE_STATE;
                                            else
                                                Next_State <= INITIAL_STATE;
                                            end if;

        end case;
end process;

WrL <= '1' when ( Current_State = SEND_STATE ) else '0';
done <= '1' when ( Current_State = DONE_STATE ) else '0';

Dout <= Din;

end behavioral;
```

A.3 SerialReceiver

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver is
        port(
        SS : in std_logic;
        MCLK : in std_logic;
        SDX : in std_logic;
        RESET : in std_logic;
        SCLK : in std_logic;
        accept : in std_logic;
        D : out std_logic_vector(4 downto 0);
        DXval : out std_logic
        );
end SerialReceiver;

architecture structural of SerialReceiver is

component SerialControl is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        enRx : in std_logic;
        accept : in std_logic;
   pFlag : in std_logic;
   dFlag : in std_logic;
        RXerror : in std_logic;
   wr : out std_logic;
        init : out std_logic;
        DXval : out std_logic
        );
end component;

component ParityCheck is
        port (
        data: in std_logic;
        init: in std_logic;
        CLK: in std_logic;
        err: out std_logic
        );
end component;

component ShiftRegister is
```

```
        port(
        data: in std_logic;
        enableShift: in std_logic;
        CLK: in std_logic;
        D: out std_logic_vector(4 downto 0)
        );
end component;

component counterSR is
        port (
        CLK, clr: in std_logic;
        Q: out std_logic_vector (2 downto 0)
        );
end component;

component comparatorSR is
        port (
        A: in std_logic_vector (2 downto 0);
        B: in std_logic_vector (2 downto 0);
        TC: out std_logic
        );
end component;

signal SR_exit : std_logic_vector(4 downto 0);
signal wr_eS : std_logic;
signal counter_exit : std_logic_vector(2 downto 0);
signal init_exit : std_logic;
signal DXval_exit : std_logic;
signal err_RX : std_logic;
signal cinco_dF : std_logic;
signal seis_pF : std_logic;

begin
U0: SerialControl port map ( RESET => RESET, CLK => MCLK, enRx => not SS, accept => accept,
pFlag => seis_pF and not SCLK, dFlag => cinco_dF and not SCLK, RXerror => err_RX, wr => wr_eS, init => init_exit,
DXval => DXval_exit );

U1: ParityCheck port map ( data => SDX, init => init_exit, CLK => SCLK, err => err_RX );

U2: ShiftRegister port map ( data => SDX, enableShift => wr_eS, CLK => SCLK, D => SR_exit );

U3: counterSR port map ( CLK => SCLK, clr => init_exit, Q => counter_exit);

U4: comparatorSR port map ( A => counter_exit, B => "101", TC => cinco_dF );

U5: comparatorSR port map ( A => counter_exit, B => "110", TC => seis_pF );

DXval <= DXval_exit;
D <= SR_exit;

end structural;
```

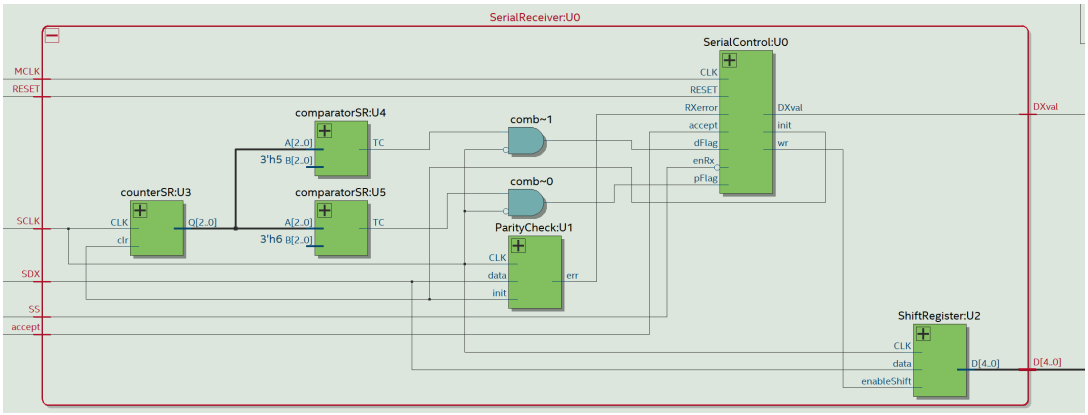

Figura 8: Netlist Viewer do *SerialReceiver*

A.4 SerialControl

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialControl is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        enRx : in std_logic;
        accept : in std_logic;
   pFlag : in std_logic;
   dFlag : in std_logic;
        RXerror : in std_logic;
   wr : out std_logic;
        init : out std_logic;
        DXval : out std_logic
        );
end SerialControl;

architecture behavioral of SerialControl is

type STATE_TYPE is (INITIAL_STATE, RECEIVING_STATE, PARITYCHECK_STATE, SENT_STATE, WAITACCEPT_STATE);

signal CurrentState, NextState : STATE_TYPE;

begin

CurrentState <= INITIAL_STATE when RESET = '1' else NextState when rising_edge(CLK);

GenerateNextState:
process (CurrentState, enRX , accept , pFlag , dFlag , RXerror)
        begin
                case CurrentState is
                        when INITIAL_STATE =>   if (enRX = '1') then
                                                        NextState <= RECEIVING_STATE;
                                                else
                                                        NextState <= INITIAL_STATE;
                                                end if;

                        when RECEIVING_STATE => if (enRX = '1' and dFlag = '1') then
                                                        NextState <= PARITYCHECK_STATE;
                                                elsif (enRX = '0') then
```

```
                                                NextState <= INITIAL_STATE;
                                        else
                                                NextState <= RECEIVING_STATE;
                                        end if;

                when PARITYCHECK_STATE        => if (enRX = '0' and pFlag = '1'and RXerror = '0') then
                                                NextState <= SENT_STATE;
                                           elsif (enRX = '1') then
                                                NextState <= PARITYCHECK_STATE;
                                           else
                                                NextState <= INITIAL_STATE;
                                           end if;

                when SENT_STATE        => if (accept = '1') then
                                        NextState <= WAITACCEPT_STATE;
                                   else
                                        NextState <= SENT_STATE;
                                   end if;

                when WAITACCEPT_STATE => if (accept = '0') then
                                                NextState <= INITIAL_STATE;
                                         else
                                                NextState <= WAITACCEPT_STATE;
                                         end if;
        end case;
end process;

init <= '1' when (CurrentState = INITIAL_STATE) else '0';
wr <= '1' when (CurrentState = RECEIVING_STATE) else '0';
DXval <= '1' when (CurrentState = SENT_STATE) else '0';

end behavioral;
```

A.5 ParityCheck

```
library ieee;
use ieee.std_logic_1164.all;

entity ParityCheck is
        port (
        data: in std_logic;
        init: in std_logic;
        CLK: in std_logic;
        err: out std_logic
        );
end ParityCheck;

architecture structural of ParityCheck is
```

```
component counterPC is
        port (
        CE, CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end component;

signal q: std_logic_vector(3 downto 0);

begin

U1: counterPC port map (CE => data ,CLK => CLK, clr => init, Q => q);

err <= not q(0);

end structural;
```

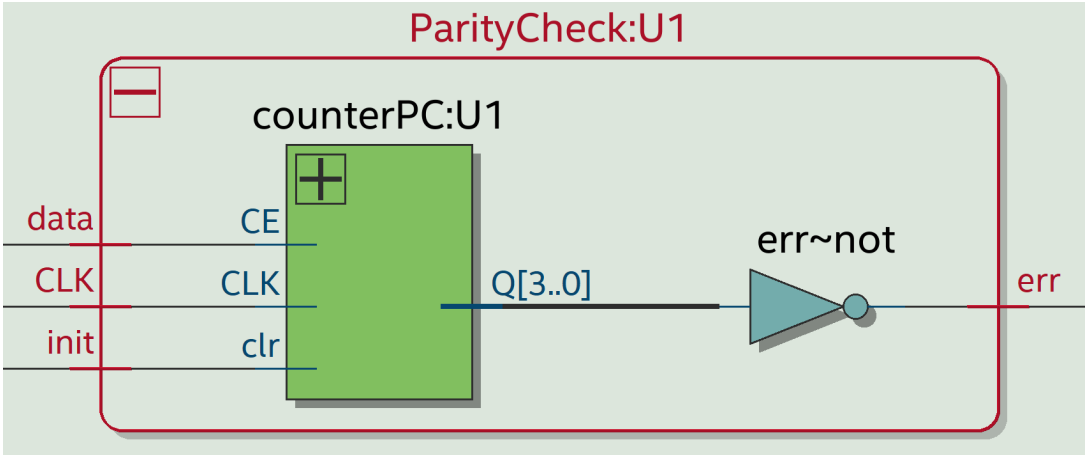

Figura 9: Netlist Viewer do *ParityCheck*

A.6 counterPC

```
library ieee;
use ieee.std_logic_1164.all;

entity counterPC is
        port (
        CE, CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end counterPC;

architecture structural of counterPC is

component adderPC is
        port(
   A, B: in std_logic_vector(3 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(3 downto 0)
   );
end component;
```

```
component regPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (3 downto 0);
signal Reg_exit: std_logic_vector (3 downto 0);

begin

U0: adderPC port map (A => Reg_exit, B => "0001", Ci => '0', Co => open, S => Adder_Reg);
U1: regPC port map (CLK => CLK, RESET => clr, D => Adder_Reg, EN => CE, Q => Reg_exit);
Q <= Reg_exit;

end structural;
```

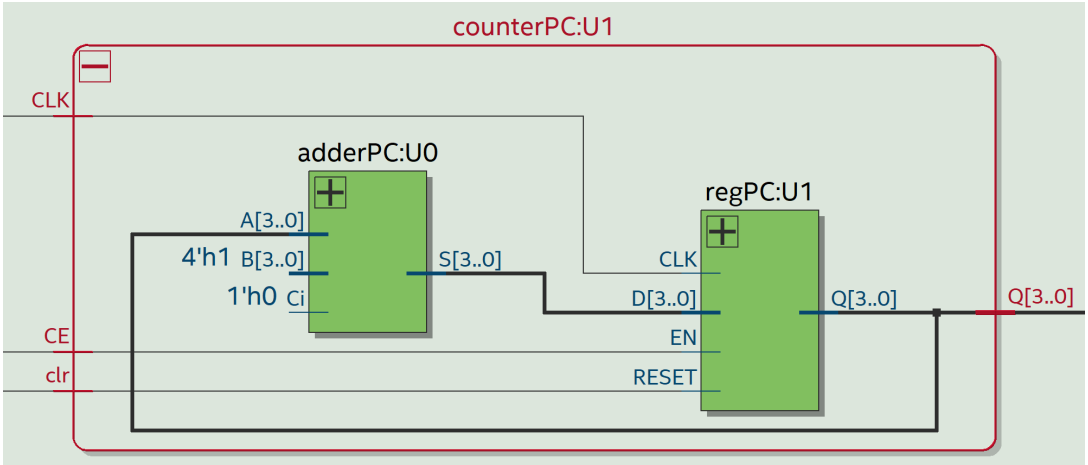

Figura 10: Netlist Viewer do *counterPC*

A.7 adderPC

```
library ieee;
use ieee.std_logic_1164.all;

entity adderPC is
        port(
        A, B: in std_logic_vector(3 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(3 downto 0);
        Co: out std_logic
        );
end adderPC;

architecture structural of adderPC is

component faPC is
        port(
```

```
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(3 downto 1);

begin

U0: faPC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: faPC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: faPC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
U3: faPC port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => Co);

end structural;
```

A.8 faPC

```
library ieee;
use ieee.std_logic_1164.all;

entity faPC is
        port(
   A, B, Ci: in std_logic;
   S, Co: out std_logic
   );
end faPC;

architecture structural of faPC is

component haPC is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: haPC port map (A => A, B => B, S => addAB, Co => carry(0));
U2: haPC port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.9 haPC

```
library ieee;
use ieee.std_logic_1164.all;

entity haPC is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end haPC;
```

```
architecture structural of haPC is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.10 regPC

```
library ieee;
use ieee.std_logic_1164.all;

entity regPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end regPC;

architecture structural of regPC is

component ffdPC is
        port(
        CLK: in std_logic;
        RESET: in std_logic;
        SET: in std_logic;
        D: in std_logic;
        EN: in std_logic;
        Q: out std_logic
        );
end component;

begin

U0: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.11 ffdPC

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdPC;

architecture structural of ffdPC is
```

```
begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.12 ShiftRegister

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftRegister is
        port(
        data: in std_logic;
        enableShift: in std_logic;
        CLK: in std_logic;
        D: out std_logic_vector(4 downto 0)
        );
end ShiftRegister;

architecture structural of ShiftRegister is

component ffdSR is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end component;

signal exitFFD: std_logic_vector(4 downto 0);

begin

U1: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => data, EN => enableShift, Q => exitFFD(0));
U2: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(0), EN => enableShift, Q => exitFFD(1));
U3: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(1), EN => enableShift, Q => exitFFD(2));
U4: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(2), EN => enableShift, Q => exitFFD(3));
U5: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(3), EN => enableShift, Q => exitFFD(4));

D(0) <= exitFFD(4);
D(1) <= exitFFD(3);
D(2) <= exitFFD(2);
D(3) <= exitFFD(1);
D(4) <= exitFFD(0);

end structural;
```

A.13 ffdSR

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdSR is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
```

```
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdSR;

architecture structural of ffdSR is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.14 counterSR

```
library ieee;
use ieee.std_logic_1164.all;

entity counterSR is
        port (
        CLK, clr: in std_logic;
        Q: out std_logic_vector (2 downto 0)
        );
end counterSR;

architecture structural of counterSR is

component adderC_SLCDC is
        port(
   A, B: in std_logic_vector(2 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(2 downto 0)
   );
end component;

component regC_SLCDC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(2 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(2 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (2 downto 0);
signal Reg_exit: std_logic_vector (2 downto 0);

begin

U0: adderC_SLCDC port map (A => Reg_exit, B => "001", Ci => '0', Co => open, S => Adder_Reg);
U1: regC_SLCDC port map (CLK => CLK, RESET => clr, D => Adder_Reg, EN => '1', Q => Reg_exit);
Q <= Reg_exit;

end structural;
```

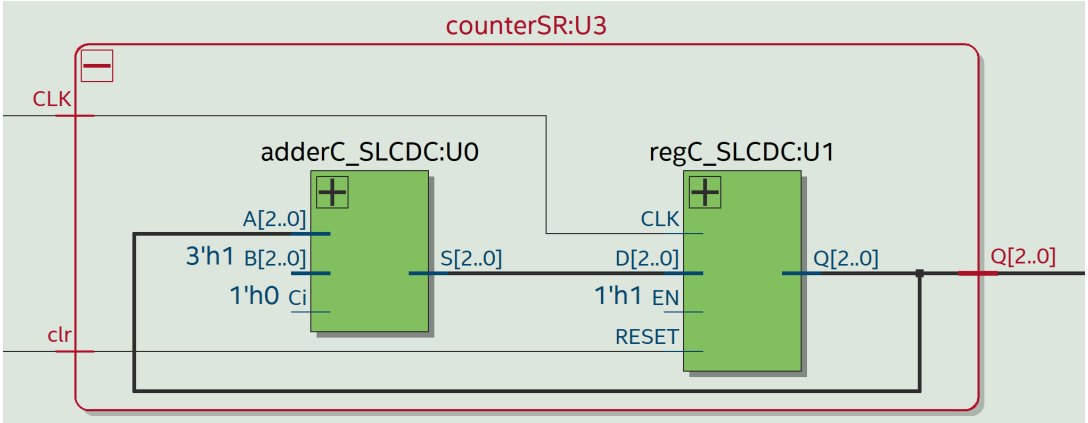

Figura 11: Netlist Viewer do *counterSR*

A.15 adderC_SLCDC

```
library ieee;
use ieee.std_logic_1164.all;

entity adderC_SLCDC is
        port(
        A, B: in std_logic_vector(2 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(2 downto 0);
        Co: out std_logic
        );
end adderC_SLCDC;

architecture structural of adderC_SLCDC is

component faC is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(2 downto 1);

begin

U0: faC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: faC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: faC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => Co);

end structural;
```

A.16 faC

```
library ieee;
use ieee.std_logic_1164.all;

entity faC is
        port(
   A, B, Ci: in std_logic;
   S, Co: out std_logic
```

```
   );
end faC;

architecture structural of faC is

component haC is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: haC port map (A => A, B => B, S => addAB, Co => carry(0));
U2: haC port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.17 haC

```
library ieee;
use ieee.std_logic_1164.all;

entity haC is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end haC;

architecture structural of haC is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.18 regC_SLCDC

```
library ieee;
use ieee.std_logic_1164.all;

entity regC_SLCDC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(2 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(2 downto 0)
        );
end regC_SLCDC;

architecture structural of regC_SLCDC is
```

```
component ffdC is
        port(
        CLK: in std_logic;
        RESET: in std_logic;
        SET: in std_logic;
        D: in std_logic;
        EN: in std_logic;
        Q: out std_logic
        );
end component;

begin

U0: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));

end structural;
```

A.19 ffdC

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdC;

architecture structural of ffdC is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.20 comparatorSR

```
library ieee;
use ieee.std_logic_1164.all;

entity comparatorSR is
        port (
        A: in std_logic_vector (2 downto 0);
        B: in std_logic_vector (2 downto 0);
        TC: out std_logic
        );
end comparatorSR;

architecture structural of comparatorSR is
begin

TC <= ((A(0) xnor B(0)) and (A(1) xnor B(1)) and (A(2) xnor B(2)));
```

```
end structural;
```

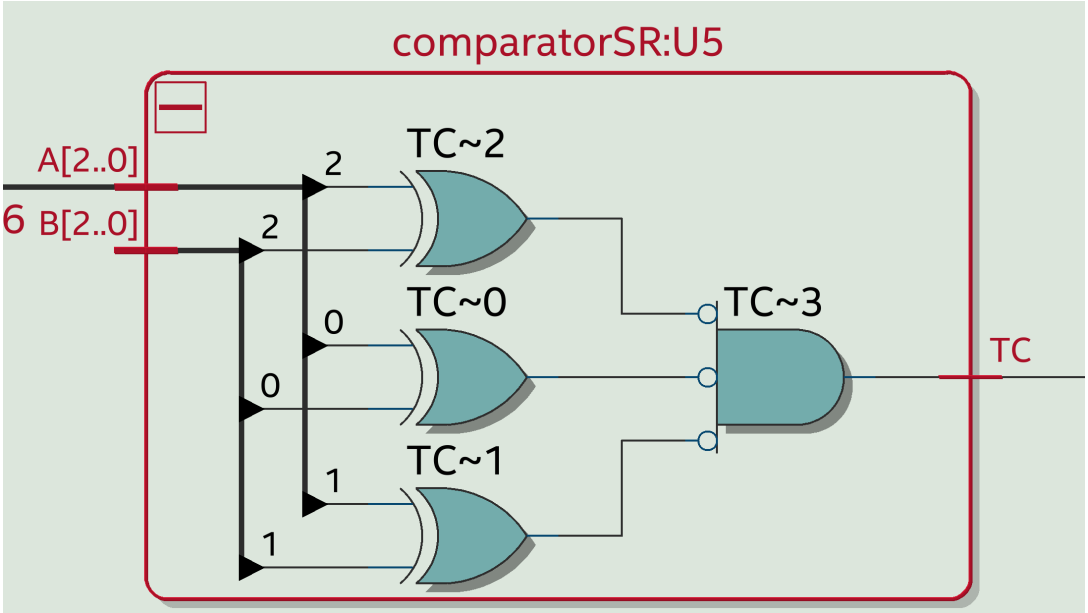

Figura 12: Netlist Viewer do *comparatorSR*

B Kotlin

B.1 HAL

Algoritmo 1: HAL

```
import isel.leic.UsbPort

object HAL {
    private var currentState = 0x00

    // Inicia o objeto
    fun init() = UsbPort.write(currentState)

    // Retorna 'true' se o bit definido pela mask esta com o valor logico '1' no UsbPort
    fun isBit(mask: Int): Boolean{
        return (UsbPort.read() and mask) != 0
    }

    // Retorna os valores dos bits representados por mask presentes no UsbPort
    fun readBits(mask: Int): Int{
        return UsbPort.read() and mask
    }

    // Escreve nos bits representados por mask os valores dos bits correspondentes em value
    fun writeBits(mask: Int, value: Int){
        clrBits(mask)
        setBits(value and mask)
    }

    // Coloca os bits representados por mask no valor logico '1'
    fun setBits(mask: Int){
        currentState = currentState or mask
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }

    // Coloca os bits representados por mask no valor logico '0'
    fun clrBits(mask: Int){
        currentState = currentState and mask.inv()
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }
}

fun main(){
    HAL.init() // Inicializa o HAL (estado inicial = 0)
    HAL.isBit(0x01) // Verifica se o bit 0 esta ativo
    HAL.readBits(0xFF) // Le o estado atual de todos os bits
    HAL.writeBits(0xF0 , 0xA0) // Escreve 1010_0000 nos bits 4 a 7 (com mascara 0xF0)
    HAL.setBits(0x0F) // Ativa os primeiros 4 bits (bits 0 a 3)
    HAL.clrBits(0x05) // Limpa os bits 0 e 2 (mascara 0000_0101)
    println("Bit 1 ativo? ${HAL.isBit(0x02)}") // Exibe o estado final do bit 1
}
```

B.2 SerialEmitter

Algoritmo 2: SerialEmitter

```
// Envia tramas para os diferentes modulos Serial Receiver.
object SerialEmitter {
    enum class Destination {LCD, ROULETTE}

    const val SDX_MASK = 0x01
    const val SCLK_MASK = 0x02
    const val LCD_SEL_MASK = 0x04
    const val P_MASK = 0x01
    const val RD_SEL_MASK = 0x08

    // Inicia a classe
    fun init () = HAL.setBits(LCD_SEL_MASK or RD_SEL_MASK)
```

```
    // Envia uma trama para o Serial Receiver
    // identificado o destino em     addr    ,
    // os bits de dados em     data
    // e em     size     o numero de bits a enviar.
    fun send(addr: Destination, data: Int, size: Int) {

        val destinationBit = if (addr == Destination.LCD) LCD_SEL_MASK else RD_SEL_MASK

        HAL.clrBits(destinationBit)

        var parity = 0

        for (i in 0 until size) {
            val bit = (data shr i) and P_MASK
            if (bit == 1) HAL.setBits(SDX_MASK) else HAL.clrBits(SDX_MASK)

            HAL.setBits(SCLK_MASK)
            HAL.clrBits(SCLK_MASK)

            parity = parity xor bit
        }

        // Envia o bit de paridade
        if (parity == 0) HAL.setBits(SDX_MASK) else HAL.clrBits(SDX_MASK)

        HAL.setBits(SCLK_MASK)
        HAL.clrBits(SCLK_MASK)

        HAL.setBits(destinationBit)
    }
}

fun main() {
    HAL.init()
    SerialEmitter.init()
    SerialEmitter.send(SerialEmitter.Destination.LCD, 0x15, 5)
}
```

B.3 LCD

Algoritmo 3: LCD

```
import isel.leic.utils.Time
import java.io.Serial

// Escreve no LCD usando a interface a 4 bits
object LCD {

    // Dimensao do display
    private const val LINES = 2
    private const val COLS = 16
    // Define se a interface e Serie ou Paralela
    private const val SERIAL_INTERFACE = true

    // Mascaras para os pinos do controlo
    private const val RS_MASK = 0x10  // Register Select (RS) - Define comando ou dado
    private const val E_MASK = 0x20   // Enable (E) - Ativa o LCD
    private const val DATA_MASK = 0x0F // Mascara para os 4 bits de dados (D4 a D7)
    private const val CGRAM_MASK = 0x40 //Mascara para escrever o caracter especial
    private const val DDRAM_MASK = 0x80

    // Escreve um byte de comando / dados no LCD em paralelo
    private fun writeNibbleParallel(rs: Boolean, data: Int) {
        if (rs) HAL.setBits(RS_MASK) else HAL.clrBits(RS_MASK)
        HAL.setBits(E_MASK)
        HAL.writeBits(DATA_MASK, data)
        HAL.clrBits(E_MASK)
        //HAL.clrBits(RS_MASK)
    }

    // Escreve um byte de comando / dados no LCD em serie
    private fun writeNibbleSerial(rs: Boolean, data: Int) {
        val nibbleWithRS = (if (rs) data.shl(1) or 1 else data.shl(1) or 0)
```

```
    //val nibbleWithRS = (if (rs) RS_MASK else 0x00) or (data and DATA_MASK)
    SerialEmitter.send(SerialEmitter.Destination.LCD , nibbleWithRS, 5)
}

// Escreve um nibble de comando / dados no LCD
private fun writeNibble(rs: Boolean, data: Int) {
    if (SERIAL_INTERFACE) {
        writeNibbleSerial(rs, data)
    } else {
        writeNibbleParallel(rs, data)
    }
}

// Escreve um byte de comando / dados no LCD
private fun writeByte(rs: Boolean, data: Int) {
    writeNibble(rs, (data shr 4) and DATA_MASK) // Envia os 4 bits mais significativos
    writeNibble(rs, data and DATA_MASK) // Envia os 4 bits menos significativos
}

// Escreve um comando no LCD
private fun writeCMD(data: Int) {
    writeByte(false, data) // RS = 0 (Comando)
}

// Escreve um dado no LCD
private fun writeDATA(data: Int) {
    writeByte(true, data) // RS = 1 (Dado)
}

// Envia a sequencia de iniciacao para comunicacao a 4 bits
fun init() {

    Time.sleep(50)

    // Inicializa  o especial (modo 8 bits temporario)
    writeNibble(false, 0x03)
    Time.sleep(5) // Espera 5ms

    writeNibble(false, 0x03)
    Time.sleep(1) // Espera 1ms

    writeNibble(false, 0x03)

    // Ativa o modo 4 bits
    writeNibble(false, 0x02)

    // Configura  o do LCD
    writeCMD(0x28) // Modo 4 bits, 2 linhas, fonte 5x8
    writeCMD(0x08) // Display Off
    writeCMD(0x01) // Limpa o display
    writeCMD(0x06) // Cursor move para a direita
    writeCMD(0x0C) // Display On
}

// Escreve um carater na posicao corrente
fun write(c: Char) {
    writeDATA(c.code) // Envia codigo ASCII do carater
}

// Escreve uma string na posicao corrente
fun write(text: String) {
    var currentLine = 0
    var currentColumn = 0

    // Escreve o texto a partir da posi ao atual do cursor
    for (i in text.indices) {
        this.write(text[i])
        currentColumn++

        // Se atingir o fim da linha (16 colunas) e ainda houver texto
        if (currentColumn >= COLS && i < text.length - 1) {
            if (currentLine == 0) { // So muda para a linha 1 se estiver na linha 0
                currentLine = 1
                cursor(currentLine, 0)
```

```
                    currentColumn = 0
                } else {
                    break // Para se ja estiver na linha 1
                }
            }
        }
    }

    // Envia comando para posicionar cursor (    line    : 0 .. LINES-1,     column    : 0 .. COLS-1)
    fun cursor(line: Int, column: Int) {
        var finalLine = line
        var finalColumn = column

        // Garante que a linha esta dentro dos limites (0 ou 1)
        if (finalLine < 0) finalLine = 0
        if (finalLine >= LINES) finalLine = LINES - 1 // Maximo e LINES-1

        // Garante que a coluna esta dentro dos limites (0 a 15)
        if (finalColumn < 0) finalColumn = 0
        if (finalColumn >= COLS) finalColumn = COLS - 1 // Maximo e COLS-1

        // Define o endere o base da linha
        val address = if (finalLine == 0) 0x80 else 0xC0

        // Move o cursor para a posi ao desejada
        writeCMD(address + finalColumn)
    }

    // Envia o comando para limpar o ecra e posicionar o cursor em (0, 0)
    fun clear( ) {
        writeCMD(0x01) // Comando para limpar o display
    }

    fun specialChar() {
        val chars = arrayOf(
            byteArrayOf(0b00000, 0b01110, 0b01110, 0b11111, 0b11111, 0b00100, 0b01110, 0b00000), // Paus
            byteArrayOf(0b00000, 0b00000, 0b01010, 0b10101, 0b10001, 0b01010, 0b00100, 0b00000), // Copas
            byteArrayOf(0b00000, 0b00100, 0b01110, 0b11111, 0b11111, 0b00100, 0b01110, 0b00000), // Espadas
            byteArrayOf(0b00000, 0b00100, 0b01010, 0b10001, 0b10001, 0b01010, 0b00100, 0b00000)
// Ouros
        )

        for ((charIndex, pattern) in chars.withIndex()) {
            writeCMD(CGRAM_MASK + (charIndex * 8))
            for (line in pattern) {
                writeDATA(line.toInt())
            }
        }
    }

    fun writeSpecialChar(index: Int) {
        writeDATA(index)
    }

    fun returnToDDRAM() {
        writeCMD(DDRAM_MASK)
    }
}


fun main() {
    HAL.init()  // Inicializa o HAL
    LCD.init() // Inicializa o LCD
    LCD.clear() // Limpa a tela
    LCD.specialChar()
    LCD.returnToDDRAM()

    LCD.write("HELLO␣WORLD!!") // Escreve uma mensagem
    LCD.cursor(1, 0) // Move o cursor para a segunda linha
    LCD.writeSpecialChar(0)
    LCD.writeSpecialChar(1)
    LCD.writeSpecialChar(2)
    LCD.writeSpecialChar(3)
}
```

C Atribuição de Pinos

```
set_global_assignment -name BOARD "MAX␣10␣DE10␣-␣Lite"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name FAMILY "MAX␣10"

#CLOCK
set_location_assignment PIN_P11 -to CLK

#RESET
set_location_assignment PIN_F15 -to RESET

#KEYPAD
set_location_assignment PIN_W5   -to KeyPadL[0]
set_location_assignment PIN_AA14 -to KeyPadL[1]
set_location_assignment PIN_W12  -to KeyPadL[2]
set_location_assignment PIN_AB12 -to KeyPadL[3]
set_location_assignment PIN_AB11 -to KeyPadC[0]
set_location_assignment PIN_AB10 -to KeyPadC[1]
set_location_assignment PIN_AA9  -to KeyPadC[2]
set_location_assignment PIN_AA8  -to KeyPadC[3]

#Dval e D
set_location_assignment PIN_A8  -to D[0]
set_location_assignment PIN_A9  -to D[1]
set_location_assignment PIN_A10 -to D[2]
set_location_assignment PIN_B10 -to D[3]
set_location_assignment PIN_D13 -to Dval

#LCD
set_location_assignment PIN_W11    -to DLCD[4]
set_location_assignment PIN_AA10   -to DLCD[5]
set_location_assignment PIN_Y8     -to DLCD[6]
set_location_assignment PIN_Y7     -to DLCD[7]
set_location_assignment PIN_W8     -to RS
set_location_assignment PIN_V5     -to E

#Display
#==========================================================
# HEX0
#==========================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[7]
set_location_assignment PIN_C14 -to HEX0[0]
set_location_assignment PIN_E15 -to HEX0[1]
set_location_assignment PIN_C15 -to HEX0[2]
set_location_assignment PIN_C16 -to HEX0[3]
set_location_assignment PIN_E16 -to HEX0[4]
```

```
set_location_assignment PIN_D17 -to HEX0[5]
set_location_assignment PIN_C17 -to HEX0[6]
set_location_assignment PIN_D15 -to HEX0[7]

#============================================================
# HEX1
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[7]
set_location_assignment PIN_C18 -to HEX1[0]
set_location_assignment PIN_D18 -to HEX1[1]
set_location_assignment PIN_E18 -to HEX1[2]
set_location_assignment PIN_B16 -to HEX1[3]
set_location_assignment PIN_A17 -to HEX1[4]
set_location_assignment PIN_A18 -to HEX1[5]
set_location_assignment PIN_B17 -to HEX1[6]
set_location_assignment PIN_A16 -to HEX1[7]

#============================================================
# HEX2
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[7]
set_location_assignment PIN_B20 -to HEX2[0]
set_location_assignment PIN_A20 -to HEX2[1]
set_location_assignment PIN_B19 -to HEX2[2]
set_location_assignment PIN_A21 -to HEX2[3]
set_location_assignment PIN_B21 -to HEX2[4]
set_location_assignment PIN_C22 -to HEX2[5]
set_location_assignment PIN_B22 -to HEX2[6]
set_location_assignment PIN_A19 -to HEX2[7]

#============================================================
# HEX3
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[6]
```

```
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[7]
set_location_assignment PIN_F21 -to HEX3[0]
set_location_assignment PIN_E22 -to HEX3[1]
set_location_assignment PIN_E21 -to HEX3[2]
set_location_assignment PIN_C19 -to HEX3[3]
set_location_assignment PIN_C20 -to HEX3[4]
set_location_assignment PIN_D19 -to HEX3[5]
set_location_assignment PIN_E17 -to HEX3[6]
set_location_assignment PIN_D22 -to HEX3[7]

#============================================================
# HEX4
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[7]
set_location_assignment PIN_F18 -to HEX4[0]
set_location_assignment PIN_E20 -to HEX4[1]
set_location_assignment PIN_E19 -to HEX4[2]
set_location_assignment PIN_J18 -to HEX4[3]
set_location_assignment PIN_H19 -to HEX4[4]
set_location_assignment PIN_F19 -to HEX4[5]
set_location_assignment PIN_F20 -to HEX4[6]
set_location_assignment PIN_F17 -to HEX4[7]

#============================================================
# HEX5
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[7]
set_location_assignment PIN_J20 -to HEX5[0]
set_location_assignment PIN_K20 -to HEX5[1]
set_location_assignment PIN_L18 -to HEX5[2]
set_location_assignment PIN_N18 -to HEX5[3]
set_location_assignment PIN_M20 -to HEX5[4]
set_location_assignment PIN_N19 -to HEX5[5]
set_location_assignment PIN_N20 -to HEX5[6]
set_location_assignment PIN_L19 -to HEX5[7]
```

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔ ...		CLK	Location	PIN_P11	Yes			
2	✔ ...		R...T	Location	PIN_F15	Yes			
3	✔ ...		Ke...0]	Location	PIN_W5	Yes			
4	✔ ...		Ke...1]	Location	PI...14	Yes			
5	✔ ...		Ke...2]	Location	PI...12	Yes			
6	✔ ...		Ke...3]	Location	PI...12	Yes			
7	✔ ...		Ke...0]	Location	PI...11	Yes			
8	✔ ...		Ke...1]	Location	PI...10	Yes			
9	✔ ...		Ke...2]	Location	PI...A9	Yes			
10	✔ ...		Ke...3]	Location	PI...A8	Yes			
11	✔ ...		Dval	Location	PI...13	Yes			
12	✔ ...		D[0]	Location	PIN_A8	Yes			
13	✔ ...		D[1]	Location	PIN_A9	Yes			
14	✔ ...		D[2]	Location	PI...10	Yes			
15	✔ ...		D[3]	Location	PIN_B10	Yes			
16	✔ ...		DL...4]	Location	PI...11	Yes			
17	✔ ...		DL...5]	Location	PI...10	Yes			
18	✔ ...		DL...6]	Location	PIN_Y8	Yes			
19	✔ ...		DL...7]	Location	PIN_Y7	Yes			
20	✔ ...		RS	Location	PIN_W8	Yes			
21	✔ ...		E	Location	PIN_V5	Yes			
22	✔ ...		HE...0]	Location	PIN_C14	Yes			
23	✔ ...		HE...1]	Location	PIN_E15	Yes			
24	✔ ...		HE...2]	Location	PIN_C15	Yes			
25	✔ ...		HE...3]	Location	PIN_C16	Yes			

Figura 13: Assignment Editor - parte 1

26	✔	...		HE...4]	Location	PIN_E16	Yes			
27	✔	...		HE...5]	Location	PI...17	Yes			
28	✔	...		HE...6]	Location	PIN_C17	Yes			
29	✔	...		HE...7]	Location	PI...15	Yes			
30	✔	...		HE...0]	Location	PIN_C18	Yes			
31	✔	...		HE...1]	Location	PI...18	Yes			
32	✔	...		HE...2]	Location	PIN_E18	Yes			
33	✔	...		HE...3]	Location	PIN_B16	Yes			
34	✔	...		HE...4]	Location	PI...17	Yes			
35	✔	...		HE...5]	Location	PI...18	Yes			
36	✔	...		HE...6]	Location	PIN_B17	Yes			
37	✔	...		HE...7]	Location	PI...16	Yes			
38	✔	...		HE...0]	Location	PIN_B20	Yes			
39	✔	...		HE...1]	Location	PI...20	Yes			
40	✔	...		HE...2]	Location	PIN_B19	Yes			
41	✔	...		HE...3]	Location	PI...21	Yes			
42	✔	...		HE...4]	Location	PIN_B21	Yes			
43	✔	...		HE...5]	Location	PIN_C22	Yes			
44	✔	...		HE...6]	Location	PIN_B22	Yes			
45	✔	...		HE...7]	Location	PI...19	Yes			
46	✔	...		HE...0]	Location	PIN_F21	Yes			
47	✔	...		HE...1]	Location	PIN_E22	Yes			
48	✔	...		HE...2]	Location	PIN_E21	Yes			
49	✔	...		HE...3]	Location	PIN_C19	Yes			

Figura 14: Assignment Editor - parte 2

50	✔ ...		HE...4]	Location	PIN_C20	Yes			
51	✔ ...		HE...5]	Location	PI...19	Yes			
52	✔ ...		HE...6]	Location	PIN_E17	Yes			
53	✔ ...		HE...7]	Location	PI...22	Yes			
54	✔ ...		HE...0]	Location	PIN_F18	Yes			
55	✔ ...		HE...1]	Location	PIN_E20	Yes			
56	✔ ...		HE...2]	Location	PIN_E19	Yes			
57	✔ ...		HE...3]	Location	PIN_J18	Yes			
58	✔ ...		HE...4]	Location	PI...19	Yes			
59	✔ ...		HE...5]	Location	PIN_F19	Yes			
60	✔ ...		HE...6]	Location	PIN_F20	Yes			
61	✔ ...		HE...7]	Location	PIN_F17	Yes			
62	✔ ...		HE...0]	Location	PIN_J20	Yes			
63	✔ ...		HE...1]	Location	PIN_K20	Yes			
64	✔ ...		HE...2]	Location	PIN_L18	Yes			
65	✔ ...		HE...3]	Location	PI...18	Yes			
66	✔ ...		HE...4]	Location	PI...20	Yes			
67	✔ ...		HE...5]	Location	PI...19	Yes			
68	✔ ...		HE...6]	Location	PI...20	Yes			
69	✔ ...		HE...7]	Location	PIN_L19	Yes			
70		<<...>>	<<...>>	<<new>>					

Figura 15: Assignment Editor - parte 3